2.1 The Game

One might say that parking at SFU is pure chaos: overcrowded lots, misplaced traffic cones, and the ever-watchful Concord parking officer. Finding a parking spot is nearly impossible, and the only way to ease this problem is by collecting together enough coins for a parking pass. Without one, you risk being forced off campus before you even have a chance to park.

In this game, players control a blue car that navigates through a maze-like parking lot filled with undesirable obstacles. The game begins at the lot entrance, where players must carefully maneuver through the chaos to gather resources and avoid penalties. Green bushes and traffic cones create barriers, though they pose no real threat – parked cars and the Concord parking officer can be deadly. These parked cars clutter the lot, and the Concord parking officer roams in search of the player. Scattered throughout are coins and lost student notes waiting to be collected.

The main objective is to collect all 10 coins, each worth 5 points, to afford a parking pass. Once players have enough, they must navigate to the exit where the ticket booth is located, completing the game. However, if their score drops below zero at any point, they immediately lose, and it is game over.

Obstacles make this game a challenge as crashing into a parked car results in a 5-point penalty, reducing the chances of surviving. Although the real danger is the Concord parking officer, constantly on patrol and will immediately kick players off campus if caught without a valid parking pass, ending the game on the spot.

Despite the risks, players can earn bonuses. Scattered throughout the lot are lost student notes, which appear randomly and disappear quickly. If players manage to grab one before it vanishes, they receive 10 bonus points.

We have remained largely faithful to our original plan. Our team consistently met after class on Mondays and Thursdays for 1-2 hours to discuss individual roles, deadlines, and tasks. During these meetings, we assigned specific responsibilities and planned the implementation of various game components. Additionally, we maintained regular communication through Discord to address concerns or clarify uncertainties regarding implementation.

To ensure steady progress, we conducted daily check-ins to assess task completion and track our adherence to deadlines. These check-ins also provided an opportunity to discuss any challenges that arose and collaborate on effective solutions as a team.

Moreover, regarding our faithfulness to our design plans, we have made some changes to the game board design and technicalities. Initially, we planned to have our walls be a concrete-like design, but switched to green bushes as this made more sense to be an outdoor parking lot instead of indoors. Another change is that we initially planned to have an 8-point reduction to the players when they collide with a parked car, but changed it to 5 points instead because this made the game easier to play, and a 5-point deduction seemed more logical.

We also removed the Cell class as it was not needed. We instead used the coordinates of the Board to access which Entity, if any, was placed on it. Also, for each Entity, a Board object was used in the constructors so that it would be easier for each Entity class to create itself onto the Board. This also allowed us to refactor our Board class, which was a large class (“God class”).

Throughout development, several changes were made to improve the game’s functionality, structure, and overall design. One significant modification was to improve user experience, we refined the end screen by adding a “Main Menu” button, giving players the option to restart or exit the game. We also enhanced visual and gameplay elements by adding methods to read and render entity images. These changes were made to optimize performance, simplify game logic, and enhance player interaction while maintaining the core mechanics of our original design.

Developing this game provided valuable insights into project management, software design, and teamwork. One of the key lessons we learned was the importance of careful planning and flexibility. While we followed our initial design closely, we had to adapt our approach when certain implementations proved inefficient or unnecessary. For example, while refactoring, we found that the Board class had unused fields and many methods that should belong in other classes. We also learned that having smaller methods and smaller classes will make it easier to test later on. It also helps to make the code more understandable and readable, instead of having large classes or spaghetti code.

Another critical takeaway was the significance of object-oriented design principles. Making certain classes like Entity an abstract class and refactoring Board, MainCharacter, and ConcordOfficer reinforced the importance of avoiding redundancy and ensuring modularity. These changes not only improved the organization of our code but also made future modifications and testing easier.

Implementing core mechanics such as collision detection, lost note generation, concord officer path finding, and UI development deepened our understanding of object-oriented design and efficient coding practices. Working with Java reinforced key programming concepts learned in class, such as inheritance, abstraction, and encapsulation, which helped us structure our code more effectively. Additionally, incorporating JUnit for testing allowed us to validate our game logic, identify potential bugs early, and ensure that key functions behaved as expected. Throughout debugging and refactoring, we gained a better appreciation for writing maintainable and scalable code, and eventually the importance of thorough testing in game development.

From a collaboration perspective, maintaining regular communication and conducting daily check-ins played a crucial role in staying on track. Discussing challenges as they arose allowed us to quickly troubleshoot issues and keep development progressing smoothly.

Overall, this project emphasized the need for both structured planning and adaptability. Moving forward, we will apply these lessons to improve our coding practices, enhance team coordination, and build more efficient game architectures.

2.2 Tutorial

In this tutorial section, we have created a video representation of our game that highlights scenarios showing the importance of the game mechanics. The link to the video is below.

Video Link:https://www.youtube.com/watch?v=gI7IDLpPFyk